
La pseudoinstrucción `la rd,inm` se traduce en `auipc` (20 bits) + `addi` (12 bits).

`auipc` suma el inmediato con el valor del PC, por lo que `addi` debe tenerlo en cuenta

Por ejemplo:

```
.data # Vamos a asumir que los datos están en 0x10000000
```

```
A: .word 1
```

```
.text # Vamos a asumir que el código está en 0x00000000
```

```
add x1, x0, x0
```

```
la x2,A
```

```
-->
```

Se traducirá en

```
auipc x2, 0x10000 # (20 bits en la parte superior de x2)
```

```
                # En el momento de ejecución de auipc, el PC vale
```

```
0x00000004
```

```
                # auipc ejecuta x2 = PC + (inm20<<12 bits), con  
que tras esta instrucción
```

```
                # x2 = 0x10000004
```

```
addi x2, x2, -4 # Aquí restamos el valor 4 para cancelar el  
efecto de la suma del PC
```

(*) Esto tiene efectos que no he tenido en cuenta en la práctica 1

La pseudoinstrucción `li rd,inm` no presenta ese problema porque se traduce en `lui+addi`, y `lui` solo utiliza el inmediato (sin PC)

– RISC-V permite que los datos de tipo `word` y `half word` residan NO alineados en memoria

Por ejemplo:

```
.data
```

```
A: .byte 0
```

```
B: .word -1 # B no está alineado a múltiplo de 4!!!
```

```
.align 4
```

```
C: .word 0
```

```
.text
```

```
la x1,A
```

```
la x2,B
```

```
lb x3,0(x1)
```

```
lw x4,0(x2)
```

```
la x5,C
```

```
sw x4,0(x5)
```

Esto tiene implicaciones sobre el tema de memoria.

En la práctica RISC-V puede soportar el acceso a datos no alineados en hardware o generar una excepción y tratarlo por software.

.text y .data no especifican direcciones.

Mila me comentaba que esto es un problema para las prácticas de cache, donde se quiere que los datos residan a partir de cierta dirección.

En Ripes se puede jugar con un .align para conseguir alguno de los efectos deseados.

Ojo: Ripes N alinea a la siguiente dirección múltiplo de N (en lugar de 2^N , como hace RISC-V)

Ejemplo:

```
.data # Asumimos que está en 0x10000000
```

```
.align 7
```

```
.byte 2
```

El byte se almacena en la dirección $0x10000005 = (268,435,461$ en decimal) porque, después de $0x10000000 = (268,435,456$ en decimal), es el siguiente valor múltiplo de 7 ($7 \times 38,347,923 = 268,435,461$)

Ripes no soporta las extensiones F/D. No podemos hacer una práctica sobre coma flotante

¿Hacer una segunda práctica sobre segmentación?

Ripes no soporta la parada de la ruta de datos para resolver dependencias porque se considera que no tiene interés en el mundo real.

Solo están las opciones de no detectar conflictos (y resolverlo mediante inserción de nop) o predict-not-taken

Además, la latencia de salto en Ripes siempre es de dos ciclos

¿Adaptamos las transparencias de la asignatura para reflejar esto?

Eso significa:

- 1) No contar parada de la ruta, directamente ir a predict-not-taken
- 2) Ir directamente a latencia de salto de 2 ciclos

¿O lo dejamos solo como un aspecto a tratar en el laboratorio? Eso tiene implicaciones de cara al examen.
